CELLULA NLP INTERNSHIP · WEEK 4

An AI Coding Assistant that knows when it doesn't know

Nagham Taweel

WHAT IT DOES

Explains code and writes code

A Streamlit chat. It decides if you want an **explanation** or **new code**, checks its knowledge base before writing, and runs the code with one click.

WHAT MAKES IT DIFFERENT

It asks instead of guessing

When its knowledge doesn't fit the request, it says so and asks the user to **teach it**. The taught solution is saved to Chroma and used next time.

HOW IT WAS CHECKED

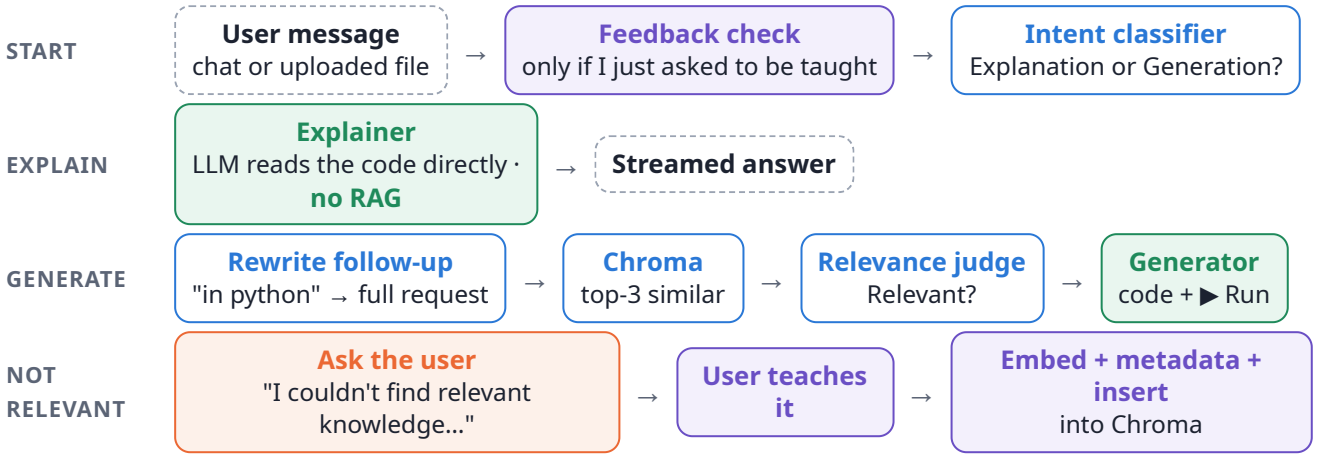
Every part tested alone

pass@1 = 85% on 20 unseen HumanEval problems. Classifier **20/20**, relevance judge **17/17**, code runner **164/164** official solutions.

144	85%	100%	164/164
HumanEval problems in the knowledge base	pass@1 on 20 held-out problems (17/20)	classifier and judge on my test sets	official solutions pass in my code runner

1 How a message travels through the system

Every "thinking" step is one LLM call (Groq, `gpt-oss-120b`, temperature 0). Everything else is plain Python.



Memory (preferences, last 3 turns, last generated code) is passed to every LLM call. **Code** only runs in a separate process when the user clicks ▶ Run.

i The project files

`app.py` is only the interface. All logic lives in `src/`, one job per file, so every piece can be tested on its own and the evaluation scripts reuse them without the UI.

`src/assistant.py` The router. Connects every part below.	**`src/classifier.py`** Explanation or Generation?	**`src/explainer.py`** Explain path (no retrieval).
`src/retriever.py` Top-k search in Chroma.	**`src/relevance_judge.py`** Relevant / Not Relevant.	**`src/generator.py`** Writes the code.
`src/feedback.py` Detects, saves and forgets taught solutions.	**`src/memory.py`** Long-term memory.	**`src/code_runner.py`** Runs code safely.
`src/knowledge_base.py` HumanEval → Chroma.	**`src/query_rewriter.py`** Short follow-up → full request.	**`prompts · config · llm · embeddings`** All prompts, settings and model setup.

2 The parts, one by one

Knowledge base: HumanEval in Chroma `knowledge_base.py`

HumanEval has 164 Python problems, each with a description, an official solution and official tests. I shuffled them with a fixed seed (42): **144 go into Chroma** and **20 are held out**, so I can later test whether the assistant writes correct code instead of copying a stored answer.

- **One document = one complete function** (signature + docstring + solution), with metadata: `task_id`, `entry_point`, `source`, `language`.
- **Kept whole, not split:** splitting would separate the description from the code. The longest problem is 1,993 characters. The splitter is still Python-aware (cuts at `def` / `class`) for long solutions users teach it.
- **Embeddings:** my Week 3 setup, `nomic-embed-text` through Ollama, with the `search_document:` / `search_query:` prefixes. Cosine distance.

Intent classifier `classifier.py`

The first LLM call for every message. To get a reliable one-label answer I used five layers:

- clear definitions of both labels, 6 examples in the prompt (few-shot), "reply with exactly one label", temperature 0;
- a **forgiving parser** that only looks for the word "explanation" or "generation", so *"Label: code explanation."* still works;
- a **safe fallback**: if the answer can't be read, use Generation, because that path checks the knowledge base and asks when unsure.

Router and explain path `assistant.py` , `explainer.py`

The router is a plain `if/else` on the label. The LLM and Chroma are loaded once, then every message calls `handle(message)`. Every path returns the same shape (`intent, route, trace, answer_stream`), so the UI shows them all the same way. The `trace` records each step and is shown under every answer.

```
intent, raw_answer = classify_intent(message, self.llm, short_memory)
trace.append(f"Classifier -> {intent}")

if intent == EXPLAIN:
    return self.explain_path(message, intent, trace)
return self.generate_path(message, intent, trace)
```

The explain path is one streamed LLM call. `explainer.py` doesn't import the retriever or embeddings at all, so it **cannot** use RAG.

Generate path: retrieval → relevance judge → generation `relevance_judge.py` , `generator.py`

Chroma **always** returns 3 results, even for a request it knows nothing about. So a second LLM call judges them before any code is written:

Request	Closest distance	Judge
Check if a string is a palindrome	0.17	**Relevant**
Sum of the even numbers in a list (not in HumanEval as-is)	0.17	**Relevant**
Create a Django model and view for blog posts	0.36	**Not Relevant**
Send an email with an attachment using SMTP	0.41	**Not Relevant**

- **Key definition:** Relevant = same language and **same kind of problem**, not the exact same problem. Otherwise every held-out or new list/string task would be Not Relevant.
- The parser checks **"not relevant" before "relevant"** (the word "relevant" is inside "not relevant"). Unreadable answer → Not Relevant: asking is safer than inventing.
- The generator writes one ```` ```python ```` block with type hints, a docstring, WHY-comments, error handling and a `__main__` demo, so ▶ Run shows output.

Human feedback learning `feedback.py`

After "Not Relevant", the request is remembered as `pending_request`. The **next** message first goes to a feedback check (an LLM call: *Solution* or *New Request*).

- If it's a solution: it's turned into a document (`"# User-taught solution for: <request>"` + the solution), given metadata (`source="user_feedback"` , `request` , `added_at`), embedded and inserted into Chroma. A copy goes to `user_solutions.jsonl` so a rebuild doesn't erase it.
- The id is a hash of request + solution, so teaching the same thing twice replaces it instead of duplicating it.
- Unsure → not saved, because a wrong "solution" would pollute the knowledge base. A wrongly saved one can be removed with `python -m scripts.forget_solution <doc_id>`.

Result: Flask API → Not Relevant → I taught it → asked again → found my solution at distance **0.22** (HumanEval was 0.40) → Relevant → Flask code generated.

Long-term memory `memory.py`

Everything is saved to `data/memory.json` (every turn, every generated code, preferences), so it survives restarts. But **only a small part goes to the LLM**: preferences + the last 3 turns + the last generated code.

The full history would quickly hit the free-tier limit of 8,000 tokens per minute.

- Preferences (language, framework) are found by whole-word keyword matching, so "java" doesn't match inside "javascript".
- Short follow-ups like *"in python"* are rewritten into a full request (`query_rewriter.py`) before searching Chroma.

Result: "sum of evens" → code; "now make it ignore values that are not integers" → the same function was modified; "explain the code you just wrote" → explained that exact code.

Code runner tool `code_runner.py`

The LLM never runs code. A separate tool receives the code as a string and returns `stdout` , `stderr` , return code and whether it timed out.

```
# A bare-minimum environment: the code can't see the API key
safe_env = {"PATH": "/usr/bin:/bin", "HOME": temp_dir}
completed = subprocess.run([sys.executable, "-I", "-X", "utf8", str(script_path)],
                           cwd=temp_dir, env=safe_env, stdin=subprocess.DEVNULL,
                           capture_output=True, text=True, timeout=timeout)
```

Safety layer	What it protects against
Runs only when the user clicks ▶ Run	Code nobody has read
Separate process + 10-second limit	Crashes and infinite loops freezing the app
Fresh temporary folder, deleted after	Files written into the project
Clean environment, isolated mode `-I`	Reading the API key (tested: it sees `None`)
No keyboard input, output cut at 10,000 characters	Hanging on `input()` , huge outputs

Not a full sandbox: the code can still read files and use the internet. Docker would close that gap (future work).

Streamlit interface `app.py`

- Chat with history, answers **streamed** piece by piece, code blocks with syntax highlighting.
- ▶ **Run code** button under generated code, with the output and errors shown below it.
- "How I answered" under every answer: the route, the retrieved documents and distances, and the judge's verdict.
- Optional extras: **file upload** (.py / .txt) and a **memory sidebar** (preferences, previous questions, generated code, what the LLM sees from memory, knowledge base size).

3 Main decisions

Decision	Choice	Why
LLM	Groq `gpt-oss-120b`, temperature 0; Ollama `llama3.1:8b` as offline fallback (one setting)	A 120B model follows "one word" rules and writes code much better than an 8B model, and it's fast.
API key	`.env` file, ignored by git	The key never appears in the code or on GitHub.
Router	Plain Python `if/else`	Only two routes; easy to read and to explain.
Embeddings	`nomic-embed-text` (from Week 3)	Already installed; requests and docstrings are both English text.
Chunking	Problems kept whole; Python-aware splitter for long texts	Keeps each description with its code.
Judge's meaning of "Relevant"	Same kind of problem, not the exact problem	Makes the assistant useful for new list/string tasks.
Feedback detection	An LLM call: Solution / New Request	"Next message is always the answer" breaks when the user moves on.
Memory	Save all, send only the recent part	Long-term memory without hitting token limits.
Code runner	Separate process with limits (not Docker)	Enough for a local tool; Docker needs hours of setup.

4 Testing

Test	What it checks	Result
Canonical solutions	All 164 official HumanEval solutions run against their own tests in my code runner. Proves the runner and test method are correct.	**164/164**
Intent classifier	20 messages (10 per label, incl. 2 in Arabic, a bug question, a "convert" request)	**20/20**
Relevance judge	17 requests: 9 HumanEval-like (incl. 2 held-out) vs. 8 outside it (Django, PyTorch, PostgreSQL, Tkinter...)	**17/17**
End-to-end, in the app	Explain · generate + run · not relevant → teach → ask again · follow-up using memory	**all worked**
pass@1 on the 20 held-out problems	The full pipeline (retrieve → judge → generate) on 20 problems that are **not** in Chroma, checked with HumanEval's official tests. Not Relevant counts as a failure. Leak check: no held-out problem was ever retrieved.	**17/20 (85%)**

What pass@1 = 85% means

17/17
every time it wrote code,
the code passed the official tests

3
misses, all because the judge
said Not Relevant and asked for help

Missed problem	Task	What Chroma found
`HumanEval/137` compare_one	Compare an int, float or a string like "2,3" and return the larger	3 problems that don't really match
`HumanEval/66` digitSum	Sum the ASCII codes of the uppercase letters in a string	3 unrelated digit/number problems
`HumanEval/124` valid_date	Check whether a date string like "03-11-2000" is valid	Nothing about dates

The system is cautious by design: when the search finds nothing close, the judge asks the user instead of writing code from unrelated examples. That's the "don't hallucinate" rule working. The trade-off is that a few problems the LLM could probably solve alone get refused. For `valid_date`, refusing is arguably right, since nothing about dates is in the knowledge base.

BEING HONEST ABOUT THE NUMBERS

The test sets are small (20 problems for pass@1), and 3 classifier messages are close to the examples in the prompt, so the 100% scores are probably optimistic.

5 Challenges and what I learned

The assistant learned something wrong

WHAT HAPPENED
I typed the follow-up "in python". Chroma was searched for just those two words → Not Relevant → my next message, a *question* with code in it, was saved as the "solution".

FIX
Short follow-ups are rewritten with memory before searching; the feedback check now knows that a question is never a solution; added a command to delete wrong knowledge.

LESSON
Testing a real conversation found a bug that one-message tests never would. A self-learning system needs an undo.

The judge couldn't decide on "sum of the even numbers in a list"

WHAT HAPPENED
Not Relevant 5 times in a row, although very similar problems were found (0.17).

FIX
Read the model's reasoning: it didn't know what "relevant" meant. Added one deciding question and examples → 5/5 Relevant.

LESSON
Unclear definitions in a prompt give unstable answers.

An English question was answered in Japanese

WHAT HAPPENED	FIX	LESSON
"Answer in the user's language" was vague when the message was mostly code.	Explicit rule: look only at the user's words; Arabic → Arabic, otherwise English.	Write rules the model can't misread.

Free-tier limits

WHAT HAPPENED	FIX	LESSON
8,000 tokens per minute caused crashes and slow answers; 200,000 tokens per day stopped the evaluations.	The client waits and retries; memory sends only a small part; one setting switches to Ollama.	Check an API's limits before planning the evaluation.

→ Limitations and next steps

- Reduce cautious misses: let the judge allow general Python tasks (like date parsing) that need no special library, then re-measure pass@1. Run the 4 automatic scenarios.
- Run the code in Docker for real isolation; add more libraries to the runner (Flask/PyTorch code currently fails to import).
- Preferences use keywords, so "don't use Flask" still records Flask; an LLM could extract them better.
- Bigger test sets for the classifier and the judge.